

WEIR - Assignment 2 - Document Retrieval

Maj Bregar, Gal Fajon, Alen Leban
Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana

Abstract—In this assignment, we created a pipeline for extracting and indexing textual content from previously crawled web pages to support domain-specific information retrieval. The pipeline parses raw HTML and retrieves semantically coherent chunks of data (article headings, body, etc.), which are then encoded into dense vector representations using a pre-trained language model. The resulting embeddings are persisted in a PostgreSQL database extended with the pgvector module, enabling efficient similarity search over the document collection.

I. INTRODUCTION

This report describes the design and implementation of a parsing and semantic embedding pipeline that processes the HTML pages collected by the preferential web crawler developed in the previous assignment. The goal is to extract clean article text from the crawled pages and represent it in a form that supports efficient domain-specific information retrieval.

The pipeline consists of two main stages: preprocessing and retrieval. During preprocessing, crawled HTML pages from 24ur.com are parsed, and the extracted text is divided into sentence-based chunks. The resulting chunks are then embedded using a transformer-based sentence embedding model and stored in a vector database.

During retrieval, a user query is embedded using the same embedding model and used to retrieve the most similar chunks from the database. Query–chunk pairs are subsequently evaluated by a reranking model, which refines the initial ranking based on semantic relevance. The final output of the pipeline is a ranked list of text chunks that are most semantically relevant to the query.

II. METHODOLOGY

A. Web Page Parsing

Parsing is implemented with the lxml library and targets the article body of each page by locating known structural elements such as the #article-body block. Article titles are recovered from OpenGraph metadata where available. The chunker uses NLTK’s Slovene sentence tokenizer to split text into sentence-boundary-aware chunks with a configurable maximum word count.

TODO: gal explain more in depth

B. Chunking

After parsing the crawled web pages, the extracted text is divided into smaller sentence-based chunks. Sentence boundaries are detected using the NLTK sentence tokenizer configured for the Slovene language. Chunks are constructed

incrementally by appending consecutive sentences until a predefined maximum word limit is reached. Once the limit is exceeded, a new chunk is created. This approach preserves local semantic context while ensuring that chunk sizes remain suitable for transformer-based embedding models.

C. Chunk Embedding

Document chunks were transformed into dense vector representations using transformer-based sentence embedding models. Each chunk was embedded independently, and the resulting vectors were stored in dedicated database tables grouped by embedding dimensionality.

To ensure consistent similarity computation during retrieval, embeddings generated by all evaluated models were normalized to unit length prior to storage. An exception was made for the BAAI/bge-m3 model, for which both normalized and unnormalized embeddings were computed in order to analyze the influence of normalization on retrieval metrics and retrieval quality.

The embedding computation included both multilingual and Slovenian-specialized embedding models. Table I summarizes the embedding models used in the experiments together with their output vector dimensionalities.

TABLE I: Embedding models used for chunk vectorization

Model	Vec. Length
sentence-transformers/all-MiniLM-L6-v2	384
sentence-transformers/all-mpnet-base-v2	768
EMBEDDIA/sloberta	768
cjvt/crosloengual-bert-si-nli	768
BAAI/bge-m3	1024

To support efficient similarity-based retrieval, vector indexes were created for each embedding table using the pgvector PostgreSQL extension. Separate HNSW indexes were generated for cosine similarity and Euclidean (L2) distance metrics, enabling experimentation with different retrieval strategies. Additionally, standard indexes were created for embedding model identifiers to enable efficient filtering by model.

D. Chunk Retrieval and Reranking

During retrieval, the user query is embedded using the same embedding model as the stored chunk embeddings. The generated query vector is then used to query the vector database and retrieve the top 100 most similar chunks according to the selected distance metric. The evaluated retrieval metrics included cosine similarity, Euclidean (L2), and Manhattan (L1) distance. Since pgvector does not provide HNSW indexing

support for L1 distance, L1 retrieval is performed without vector indexing.

The retrieved chunks are subsequently reranked using a cross-encoder model. For reranking, query–chunk text pairs are constructed and processed jointly by the reranker model to produce relevance scores. The final retrieval output consists of the 10 highest-ranked chunks according to the reranker scores.

Table II summarizes the reranking models used in the experiments.

TABLE II: Reranking model names

Reranker Model
cross-encoder/ms-marco-MiniLM-L6-v2
BAAI/bge-reranker-v2-m3

III. EMBEDDING ANALYSIS

To evaluate the retrieval quality of our setups, we employ the Normalized Discounted Cumulative Gain metric at k retrieved chunks ($\text{NDCG}@k$). This metric is well suited for retrieval pipelines, as it measures not only the number of relevant chunks among the top k retrieved results, but also whether the most relevant chunks appear near the top of the ranking. For evaluation, we compute the mean $\text{NDCG}@k$ across multiple queries, providing a more representative measure of average retrieval performance.

We first evaluated the quality of the embedding models by retrieving only the k chunks most similar to each query. This provides a baseline performance measure for selecting the best-performing embedding model without introducing bias from reranking models.

The embedding model performance is shown in Figure 1. The results indicate that the model `some-model` consistently outperforms the other evaluated models. Cosine similarity was used for all similarity-based retrieval experiments.

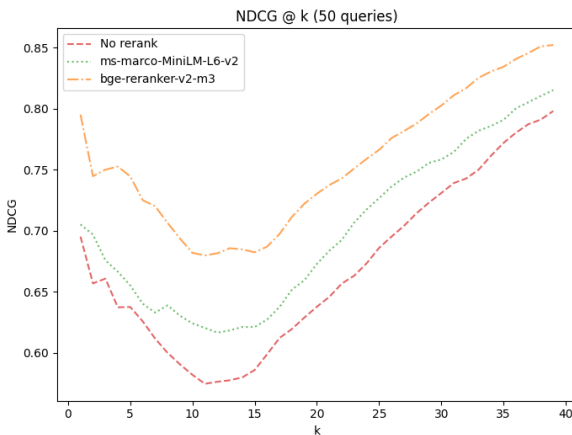


Fig. 1: Comparison of embedding models using the mean $\text{NDCG}@k$ metric over 50 queries.

Similarly, we evaluated the performance of the reranking models by first retrieving the N most similar embeddings and

then selecting the top k chunks using the reranker model. For this evaluation, the performance of the embedding model `some-model` was used as the quality baseline. As shown in Figure 2, reranking significantly improves retrieval quality, with the model `some-retriever` achieving the best performance.

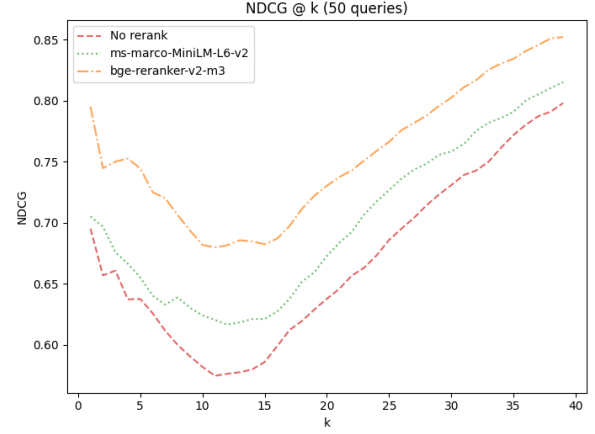


Fig. 2: Comparison of reranking models using the mean $\text{NDCG}@k$ metric over 50 queries.

Finally, we measured the impact of the used distance metric when calculating embedding similarity. Since some models normalize their embeddings by default, we used only `sentence-transformers/all-MiniLM-L6-v2` for our experiments, allowing us to use both normalized and unnormalized embeddings.

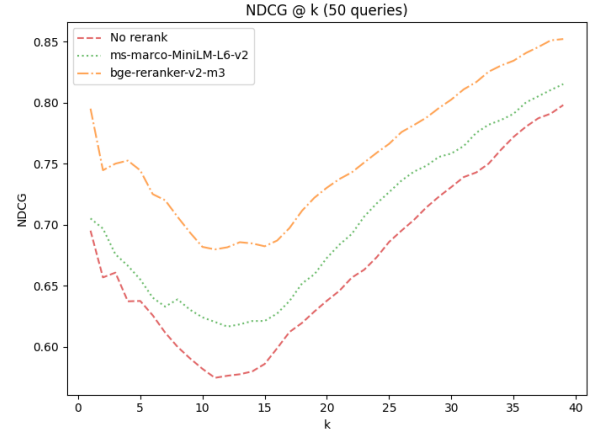


Fig. 3: Comparison of distance metric retrieval quality scored by the NDCG metric averaged over 50 different queries.

To evaluate how effectively the models group embedded chunks, we visualized chunk embeddings using UMAP. The embeddings were first reduced with SVD, retaining 40% of the variance, and then projected into a two-dimensional space

using UMAP. Each point in the scatter plot represents a single chunk embedding.

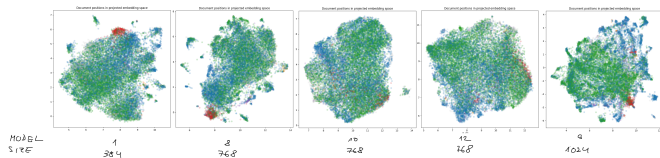


Fig. 4: UMAP visualization of chunk embedding clusters.

IV. RESULTS

The final selected parameters of our pipeline are noted in Table III.

TABLE III: Final pipeline parameters

Parameter	Chosen Value
Embedding Model	BAAI/bge-m3
Reranking Model	bge-reranker-v2-m3
Chunk Max Length	128 words
Retrieval Dist. Metric	cosine
Similarity Return n	100
Reranking Return k	10

- NDCG @ k metric for final params - examples of parsed html - examples of chunks - examples of successful queries - examples of unsuccessful queries

V. DISCUSSION

- what can be done better, what we could expand on

VI. CONCLUSION

- tmp - [1]

REFERENCES

- [1] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, vol. 4, no. 5, 2024.